

✦✦ Explicar

Pregunta **1**Sin responder
aúnSe puntúa
como 0 sobre
10,00

Complete la sección de configuración utilizando **acceso directo a registros** (Bare Metal), de modo que el pin **PC13** quede configurado como **salida push-pull de 2 MHz**. Y complete el contenido del **while** para hacer parpadear el LED conectado a **PC13 usando un loop for** para generar el retardo.

```
#define RCC_BASE 0x40021000

#define GPIOC_BASE 0x40011000

#define RCC_APB2ENR (*(volatile uint32_t *) (RCC_BASE + 0x18))
#define GPIOC_CRH (*(volatile uint32_t *) (GPIOC_BASE + 0x04))
#define GPIOC_ODR (*(volatile uint32_t *) (GPIOC_BASE + 0x0C))
#define RCC_IOPCEN (1 << 4) #define GPIOC13 (1UL << 13)

int main(void) {

/* ===== Configuración ===== */

//

//

/* ===== */

while (1) {

/* ===== parpadeo del LED ===== */

//

//

/* ===== */

} }
```

Editar Ver Insertar Formato Herramientas Tabla Ayuda

↶ ↷ **B** *I* 🔗 🔗✖️ 📐 ≡ ≡ ≡ ⌵ ⌴

≡ ≡ ⋮ ⋮₃ 📊

p

0 palabras Build with  tinyMCE

Se pontua
como 0 sobre
10,00

```
#include "stm32f1xx.h"

int main(void) {

    /* ===== Configuración ===== */

    //

    //

    /* ===== */

    while (1) {

        /* ===== parpadeo del LED ===== */

        //

        //

        /* ===== */

    } }

```




















0 palabras Build with  tinyMCE //

Pregunta **3**

Sin responder
aún

Se puntúa
como 0 sobre
10,00

Complete la sección de configuración y el manejador de interrupción para detectar la pulsación de un pulsador conectado al pin **PA0**. Usar acceso directo a registros.

El pin deberá configurarse como:

- Entrada digital con resistencia interna **pull-down**.
- Fuente de la línea de interrupción externa **EXTI0**.
- Interrupción por **flanco ascendente**.
- Interrupción EXTI0 habilitada en el **NVIC**.

Cada vez que se presione el pulsador, el manejador deberá cambiar el estado del LED conectado a **PB13**. (PB13 ya se encuentra configurado como salida)

Se dispone de las siguientes definiciones:

```
#define RCC_APB2ENR (*(volatile uint32_t *) (0x40021000 + 0x18))

//GPIOA

#define GPIOA_CRL (*(volatile uint32_t *) (0x40010800 + 0x00))
#define GPIOA_ODR (*(volatile uint32_t *) (0x40010800 + 0x0C))

//GPIOB

#define GPIOB_ODR (*(volatile uint32_t *) (0x40010C00 + 0x0C))

//AFIO

#define AFIO_EXTICR1 (*(volatile uint32_t *) (0x40010000 + 0x08))

//EXTI

#define EXTI_IMR (*(volatile uint32_t *) (0x40010400 + 0x00))
#define EXTI_RTSR (*(volatile uint32_t *) (0x40010400 + 0x08))
#define EXTI_FTSR (*(volatile uint32_t *) (0x40010400 + 0x0C))
#define EXTI_PR (*(volatile uint32_t *) (0x40010400 + 0x14))

//NVIC

#define NVIC_ISER0 (*(volatile uint32_t *) (0xE000E100))

//mascaras

#define RCC_AFIOEN (1UL << 0)
#define RCC_IOPAEN (1UL << 2)
#define GPIOA0 (1UL << 0)
#define GPIOB13 (1UL << 13)
#define EXTI0_IRQN 6

void EXTI0_IRQHandler(void);

int main(void) {

/* ===== Configuración de PA0 y EXTI0 ===== */

//

//
```

```
/* ===== */  
while (1) {  
/* programa principal */  
} }  
void EXTI0_IRQHandler(void)  
{  
/* ===== Manejador de interrupcion ===== */  
  
//  
  
//  
/* ===== */  
}
```

Editar Ver Insertar Formato Herramientas Tabla Ayuda



p

0 palabras Build with  tinyMCE //

Pregunta **4**Sin responder
aúnSe puntúa
como 0 sobre
10,00

Complete la sección de configuración y el manejador de interrupción para detectar la pulsación de un pulsador conectado al pin **PA0**. Usar las estructuras, máscaras de bits y funciones de **CMSIS**.

El pin **PA0** deberá configurarse como:

- Entrada digital con resistencia interna **pull-down**.
- Fuente de la línea de interrupción externa **EXTI0**.
- Interrupción por **flanco ascendente**.
- Interrupción EXTI0 habilitada en el **NVIC**.

Cada vez que se presione el pulsador, el manejador deberá cambiar el estado del LED conectado a **PB13**. (PB13 ya se encuentra configurado como salida)

Se dispone de las siguientes definiciones:

```
#include "stm32f1xx.h"
#include <stdint.h>

void EXTI0_IRQHandler(void);

int main(void) {

    /* ===== Configuración de PA0 y EXTI0 ===== */

    //

    //

    /* ===== */

    while (1) {

        /* programa principal */

    } }

void EXTI0_IRQHandler(void)

{

    /* ===== Manejador de interrupcion ===== */

    //

    //

    /* ===== */

}
```

Editar Ver Insertar Formato Herramientas Tabla Ayuda



p

0 palabras Build with  tinyMCE //

Pregunta **5**

Sin responder aún

Se puntúa como 0 sobre 10,00

Complete las funciones `systick_init_ms(void)`, `SysTick_Handler(void)`, `delay_ms(int)` para implementar una base de tiempos de **1 ms** utilizando el temporizador **SysTick**.

Considere que:

- La frecuencia del procesador es **72 MHz**.
- El reloj del SysTick utiliza como fuente **HCLK**.
- Cada interrupción del SysTick debe producirse cada **1 ms**.

Se dispone de las siguientes definiciones:

```
#define SysTick_CTRL (*(volatile uint32_t *) (0xE000E010))
#define SysTick_LOAD (*(volatile uint32_t *) (0xE000E014))
#define SysTick_VAL (*(volatile uint32_t *) (0xE000E018))
#define SysTick_CTRL_ENABLE (1 << 0)
#define SysTick_CTRL_TICKINT (1 << 1)
#define SysTick_CTRL_CLKSOURCE (1 << 2)

void systick_init_ms(void) {
    /* ===== Configuración ===== */
    //
    //
    /* ===== */
}

void SysTick_Handler(void) {
    /* ===== */
    //
    /* ===== */
}

void delay_ms(int time) {
    /* ===== */
    //
    /* ===== */
}
```

Editar Ver Insertar Formato Herramientas Tabla Ayuda



p

0 palabras Build with  tinyMCE //

Pregunta **6**

Sin responder aún

Se puntúa como 0 sobre 10,00

Complete las funciones `systick_init_ms(void)`, `SysTick_Handler(void)`, `delay_ms(int)` para implementar una base de tiempos de **1 ms** utilizando el temporizador **SysTick** con **CMSIS**.

Considere que:

- La frecuencia del procesador es **72 MHz**.
- La variable `SystemCoreClock` contiene dicha frecuencia.

Se dispone de las siguientes definiciones:

```
#include "stm32f1xx.h"

uint32_t SysTick_Config(uint32_t ticks);

void systick_init_ms(void) {
    /* ===== Configuración ===== */
    //
    //
    /* ===== */
}

void SysTick_Handler(void) {
    /* ===== */
    //
    /* ===== */
}

void delay_ms(int time) {
    /* ===== */
    //
    /* ===== */
}
```

Editar Ver Insertar Formato Herramientas Tabla Ayuda



p

0 palabras Build with  tinyMCE 

Pregunta **7**

Sin responder
aún

Se puntúa
como 0 sobre
10,00

Se desea medir una señal analógica conectada al **canal 2 del ADC1**, para lo cual se pide completar la función `adc_init()` y el bucle principal para que el programa:

- Configure el pin correspondiente al canal 2 como **entrada analógica**.
- Habilite los relojes necesarios para utilizar el GPIO y el ADC1.
- Configure el tiempo de muestreo del canal 2.
- Encienda el ADC1.
- Lea el canal 2 cada **500 ms**. (Usar `delay_ms(int ms)` no implementar)
- Escriba la función `adc_read(int canal)` para la lectura. Que recibe el número de canal y devuelve el resultado de la conversión.

Complete únicamente las secciones solicitadas:

```
#include "stm32f1xx.h"

#include <stdint.h>

void adc_init(void);

int adc_read(int canal);

void delay_ms(int tiempo);

int main(void) {
    uint16_t valor_adc;

    adc_init();

    while (1) {

        /*===== Leer el canal 2 y esperar 500 ms ===== */

        //

        //

        /*=====*/

    } }

void adc_init(void) {

    /*===== Configurar el GPIO y el ADC1 ===== */

    //

    //

    /*=====*/

}

int adc_read(int canal) {

    /* ===== Lectura ADC ===== */

    //

    //

    /* ===== */

}
```

Editar Ver Insertar Formato Herramientas Tabla Ayuda



p

0 palabras Build with tinyMCE //

Pregunta **8**

Sin responder
aún

Se puntúa
como 0 sobre
15,00

Se desea generar una señal PWM sobre la salida **TIM2_CH3 (PA2)** para controlar el brillo de un LED.

Complete la función `pwm_init()` y el programa principal para que:

- Configure **PA2** como salida de función alternativa *push-pull*.
- Habilite los relojes necesarios.
- Configure el temporizador **TIM2** para generar una señal PWM de **1 kHz**.
- Inicialice el ciclo de trabajo (duty cycle) en **10 %**.
- En el programa principal modifique el duty cycle entre **10 %**, **50 %** y **90 %**, manteniendo cada valor durante **1 segundo**.

Considere que:

- El reloj del temporizador TIM2 es de **72 MHz**.
- La frecuencia del PWM viene dada por:

$$f_{PWM} = \frac{f_{TIM}}{(PSC + 1)(ARR + 1)}$$

- La función `delay_ms(uint32_t tiempo)` ya se encuentra implementada (no implementar).

Complete únicamente las secciones solicitadas:

```
#include "stm32f1xx.h"

#include <stdint.h>

void pwm_init(void);

void delay_ms(uint32_t ms);

int main(void) {

    pwm_init();

    while (1) {

        /*===== Duty Cicle de 10%, 50% y 90% cada 1s ===== */

        //

        //

        /*===== */

    } }

void pwm_init(void) {

    /*===== Configurar el GPIOA y el TIM2 ===== */

    //

    //

    /*===== */

}
```



p

0 palabras Build with  tinyMCE //

Pregunta **9**

Sin responder
aún

Se puntúa
como 0 sobre
15,00

Se desea establecer una comunicación serie entre un microcontrolador STM32F103 y una PC utilizando el periférico **USART1**.

Complete la función `usart1_init()` y las funciones de transmisión y recepción para que:

- Configure **PA9** como salida de función alternativa *push-pull* (TX).
- Configure **PA10** como entrada flotante (RX).
- Habilite los relojes necesarios.
- Configure USART1 para una velocidad de **19200 baudios**.
- Configure la comunicación con formato **8N1**.
- Habilite el transmisor, el receptor y el periférico USART1.

Considere que:

- USART1 recibe un reloj **PCLK2 = 72 MHz**.
- El valor del registro **BRR** debe calcularse a partir de la velocidad de transmisión indicada.
- La transmisión y recepción se realizarán mediante **polling**.
- No se utilizarán interrupciones.

Complete únicamente las siguientes funciones:

```
#include "stm32f1xx.h"

#include <stdint.h>

void usart1_init(void) {
    /*===== Configurar el USART1 ===== */
    //
    //
    /*=====*/
}

void usart1_send_char(char dato) {
    /*===== Transmisión de dato ===== */
    //
    //
    /*=====*/
}

char usart1_receive_dato(void) {
    /*===== Recepción dato ===== */
    //
    //
    /*=====*/
}
```


Editar Ver Insertar Formato Herramientas Tabla Ayuda



p

0 palabras Build with tinyMCE //

[◀ Entrega: TP2, ejercicio 5](#)

[Saltar a actividad](#)

[Clases Teórico 2025 ▶](#)